

EXPERIMENT 1

Data Preprocessing and Feature Engineering

(Handling missing values, normalisation, encoding categorical variables, and visualization)

Aim

To perform data preprocessing and feature engineering by handling missing values, scaling numerical features, encoding categorical variables, detecting outliers, and visualizing the dataset in order to improve data quality and prepare the dataset for effective machine learning model development.

Theory

Data Preprocessing

In machine learning, raw datasets collected from real-world environments often contain inconsistencies such as missing values, categorical attributes, noisy data, and varying feature scales. These issues can significantly affect the performance and reliability of machine learning algorithms.

Data preprocessing is therefore a fundamental step that involves cleaning, transforming, and organizing the dataset before it is used for analysis or model training. Proper preprocessing ensures that the dataset is complete, consistent, and structured in a way that allows machine learning algorithms to learn meaningful patterns from the data.

Handling Missing Values (Numerical Features)

Missing values occur when observations for certain attributes are absent in a dataset. This may happen due to incomplete data collection, measurement errors, or system failures. Numerical attributes such as age, fare, or income frequently contain missing values that must be addressed before analysis.

Several statistical imputation techniques are commonly used to replace missing numerical values.

Mean Imputation replaces missing values with the average value of the feature. If a numerical attribute X contains missing values, the mean value is calculated as:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The missing values are then replaced with $\bar{x}$.

Median Imputation replaces missing values with the median of the feature, which represents the middle value when the data is sorted. This approach is particularly useful when the data distribution is skewed or contains outliers.

Mode Imputation replaces missing values with the most frequently occurring value in the feature. Although more common for categorical attributes, it can also be used for numerical variables with repeated values.

Constant Imputation replaces missing values with a fixed constant such as zero or another predefined value. This approach is sometimes used when missing values need to be represented explicitly.

Handling numerical missing values ensures that machine learning algorithms receive complete data without introducing bias or distortion in the dataset.

Handling Missing Values (Categorical Features)

Categorical attributes represent qualitative information such as labels, names, or categories. Missing values in categorical variables are commonly handled using most frequent value imputation, also known as mode imputation.

In this technique, the missing values are replaced with the category that appears most frequently in the dataset. This method helps preserve the distribution of categorical values while maintaining the consistency of the dataset.

For example, if the most frequent value in the “Embarked” column is “S”, all missing entries in that column can be replaced with “S”. This approach ensures that the dataset remains logically consistent without introducing new artificial categories.

Encoding categorical features

Many machine learning algorithms require numerical input and cannot process textual categorical values directly. Categorical encoding converts categorical variables into numerical representations while preserving their meaningful structure.

Several encoding techniques are commonly used.

Label Encoding assigns a unique integer value to each category. For example:

Category	Encoded Value
Male	0
Female	1

This technique is commonly used for binary categorical variables.

One-Hot Encoding converts each category into a separate binary feature. For example, if a variable has categories **A, B, and C**, it is transformed into three separate columns where each column indicates whether a particular category is present or not.

A	B	C
True	False	False
False	True	False
False	False	True

In this representation, True indicates the presence of a category, while False indicates its absence. This method ensures that no artificial ordering is introduced between categories. One-Hot Encoding is particularly suitable for nominal variables, where the categories do not have any inherent ranking or order.

Ordinal Encoding is used when categories have a natural ranking. Categories are assigned numerical values based on their relative order. For example:

Pclass	Encoded
1	0
2	1
3	2

This encoding preserves the hierarchical relationship between categories.

Selecting the correct encoding technique ensures that categorical features are represented accurately without introducing misleading relationships.

Feature Scaling and Normalization

In many datasets, numerical features have significantly different ranges. Machine learning algorithms that rely on distance calculations can become biased toward features with larger magnitudes. Feature scaling addresses this issue by transforming numerical features into comparable ranges.

Three common scaling techniques are used.

Min-Max Scaling

Min-Max scaling rescales data into a fixed range, typically between 0 and 1. The transformation is performed using the formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Where:

- X = original value
- X_{min} = minimum value of the feature
- X_{max} = maximum value of the feature

This method preserves the relationships among original data values but may be sensitive to outliers.

Standard Scaling (Standardization)

Standard scaling transforms the dataset so that the feature has a mean of zero and a standard deviation of one.

The transformation is given by:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = original value
- μ = mean of the feature
- σ = standard deviation

Standardization is widely used when data follows an approximately normal distribution.

Robust Scaling

Robust scaling is designed to handle datasets containing extreme values or outliers. Instead of using the mean and standard deviation, it uses the median and the interquartile range (IQR).

The transformation is defined as:

$$X_{scaled} = \frac{X - Median}{IQR}$$

Where:

$$IQR = Q3 - Q1$$

This method is less affected by extreme values and is suitable for datasets containing significant outliers.

Outlier Detection

Outliers are observations that significantly deviate from the majority of data points in a dataset. These extreme values can distort statistical analysis and negatively impact machine learning models.

Two commonly used techniques for detecting outliers are the Z-Score method and the Interquartile Range (IQR) method.

Z-Score Method

The Z-Score measures how many standard deviations a data point lies from the mean of the dataset.

The formula for the Z-Score is:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = observed value
- μ = mean of the dataset
- σ = standard deviation

Typically, any value with a Z-Score greater than ± 3 is considered an outlier.

Interquartile Range (IQR) Method

The IQR method is based on the concept of quartiles, which divide the dataset into four equal parts.

- Q1 (First Quartile) → 25th percentile
- Q2 (Median) → 50th percentile
- Q3 (Third Quartile) → 75th percentile

The interquartile range is defined as:

$$IQR = Q3 - Q1$$

Outliers are identified using the following boundaries:

$$\begin{aligned} \text{Lower Bound} &= Q1 - 1.5 \times IQR \\ \text{Upper Bound} &= Q3 + 1.5 \times IQR \end{aligned}$$

Values outside this range are considered potential outliers.

The IQR method is robust because it relies on quartiles rather than the mean, making it less sensitive to extreme values.

Feature Engineering

Feature engineering is the process of transforming existing attributes or creating new features from the available data in order to better represent the underlying patterns within a dataset. Instead of relying solely on raw variables, feature engineering helps improve the quality and relevance of the input features used by machine learning algorithms.

Feature engineering may involve combining multiple attributes, deriving new variables from existing features, or transforming attributes into more meaningful representations. By capturing relationships between different variables, engineered features can provide additional contextual information that may not be directly available in the original dataset.

For example, multiple related attributes describing similar characteristics can be combined to form a single informative feature that summarizes their combined effect. Such transformations help simplify the dataset while preserving important information.

Effective feature engineering enhances the expressive power of the dataset and allows machine learning models to identify meaningful patterns more accurately. As a result, well-designed engineered features often lead to improved model performance, better interpretability, and more reliable predictions.

Data Visualization

Data visualization plays an important role in exploratory data analysis by providing graphical representations of data. Visual techniques help identify patterns, trends, distributions, and anomalies within the dataset.

Univariate visualizations analyze the distribution of individual features, while bivariate visualizations help examine relationships between pairs of variables.

Visualization also helps validate preprocessing steps such as normalization, encoding, and outlier detection. By observing changes in distributions and patterns, analysts can confirm that preprocessing techniques have been applied correctly.

Through intuitive graphical representations, visualization enables a deeper understanding of the dataset and supports better decision-making throughout the machine learning workflow.



Fig 1: End-to-End Data Preprocessing and Feature Engineering Pipeline

The figure 1 illustrates the pipeline of the experiment, showing the sequence of steps involved in data preprocessing and feature engineering, starting from raw data and resulting in preprocessed data ready for machine learning models.

Procedure

In this experiment we will perform data preprocessing and feature engineering on the Titanic dataset by handling missing values, encoding categorical variables, normalizing numerical features, creating new meaningful features, and visualizing the data, in order to improve data quality and prepare it for effective machine learning model development.

Step 1: Importing Required Libraries

The required Python libraries are imported at the beginning of the program. These include NumPy and Pandas for data manipulation and handling, Matplotlib and Seaborn for data visualization, and Scikit-learn utilities such as *LabelEncoder*, *OrdinalEncoder*, *StandardScaler*, *MinMaxScaler*, *RobustScaler*, and *z-score functions* for preprocessing operations like encoding, feature scaling, and outlier detection.

Additionally, IPython display utilities (HTML display) and ipywidgets are used to build an interactive simulation environment, enabling dynamic user inputs and real-time visualization of preprocessing steps.

Step 2: Loading the Dataset

The Titanic dataset is loaded from a CSV file using the **read_csv()** function of the Pandas library. The dataset is stored in a DataFrame structure for further analysis. After loading, the dataset contains **891 rows** and multiple columns representing passenger-related attributes.

Step 3: Displaying Initial Records

The first few rows of the dataset are displayed using the **head()** function. This provides an initial overview of the dataset and helps in understanding the structure and values of the attributes.

Step 4: Dataset Analysis

Basic dataset analysis is performed using the **info()** function to examine:

- Column names
- Data types of each attribute
- Number of non-null values in each column

This step helps distinguish between **numerical and categorical variables** and provides an initial indication of missing values present in the dataset.

Displaying Descriptors

In addition to structural information, **descriptive statistics** of the dataset are examined using statistical summary functions. These descriptors provide insights into measures such as **mean, standard deviation, minimum, maximum, and quartile values**, which help in understanding the distribution of numerical attributes.

Step 5: Identifying Missing Values

Missing values in the dataset are identified by calculating the number of null entries present in each column. This helps determine which attributes require missing value treatment before further preprocessing steps are applied.

Step 6: Handling Missing Values (Numerical Attributes)

Missing values in numerical attributes are handled using statistical imputation techniques. The **Age** attribute is treated using different imputation methods such as:

- Mean imputation
- Median imputation
- Mode imputation
- Constant value imputation

These methods replace missing values with meaningful statistical estimates so that the dataset remains complete for further analysis.

Step 7: Handling Missing Values (Categorical Attributes)

Missing values in categorical attributes are handled using the **most frequent value method**. The category that occurs most frequently within the feature is identified and used to replace missing entries. This method helps maintain the natural distribution of categorical data while ensuring dataset completeness.

Step 8: Encoding Categorical Variables

Categorical variables are transformed into numerical representations to make them compatible with machine learning algorithms. The following encoding techniques are applied depending on the nature of the categorical attribute:

- **Label Encoding** for binary categorical variables
- **One-Hot Encoding** for nominal variables without inherent ordering
- **Ordinal Encoding** for categorical attributes that possess a natural ranking

This step ensures that categorical features can be effectively used during model training and analysis.

Step 9: Feature Scaling (Normalization)

Numerical attributes such as **Age** and **Fare** are scaled to a comparable range using feature scaling techniques. Scaling ensures that features with larger magnitudes do not dominate the learning process.

The following scaling techniques are explored:

- **StandardScaler (Standardization)**
- **MinMaxScaler (Min-Max Normalization)**
- **RobustScaler (Scaling using median and interquartile range)**

Feature scaling improves numerical stability and helps machine learning algorithms converge more efficiently.

Step 10: Outlier Detection

Outliers present in numerical attributes are identified using statistical techniques such as:

- **Z-Score Method**
- **Interquartile Range (IQR) Method**

These techniques help detect extreme values that significantly deviate from the majority of the dataset. Identifying outliers assists in understanding unusual patterns and improving data reliability.

Step 11: Feature Engineering

Feature engineering is performed to generate new informative attributes from the existing dataset. This step involves transforming or combining relevant attributes to create features that better represent relationships within the data. Such engineered features enhance the dataset's ability to capture meaningful patterns during analysis.

Step 12: Data Visualization and Analysis

Data visualization techniques are used to explore the dataset and analyze feature distributions. Graphical representations such as **histograms, box plots, and categorical plots** are used to study the distribution of attributes, identify patterns, and observe the presence of anomalies or outliers.

Visualization also helps validate preprocessing steps such as scaling, encoding, and missing value handling.

Step 13: Final Dataset Verification { raw vs processed }

The final preprocessed dataset is examined using descriptive statistics and summary functions to ensure that:

- Missing values have been properly handled
- Categorical variables are correctly encoded
- Numerical attributes are scaled appropriately
- Engineered features are successfully generated
- Outliers have been analyzed

This step confirms that the dataset is clean, consistent, and ready for further machine learning tasks.

Step 1: Importing Libraries

Importing Required Libraries

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler, RobustScaler, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression

import ipywidgets as widgets
from IPython.display import display
from scipy.stats import zscore
from IPython.display import display, HTML, clear_output
from matplotlib.pyplot import summer
print("Libraries imported")
```

Output:

Libraries Imported

Step 2: Loading Dataset

Load the Titanic dataset

```
file_path = "Titanic-Dataset.csv"
data = pd.read_csv(file_path)
print("Dataset loaded successfully")
```

Output:

Dataset loaded successfully

Convert Age to integer

```
df_raw = pd.read_csv("Titanic-Dataset.csv")
df_clean = df_raw.copy()
df_clean["Age"] = df_clean["Age"].round().astype("Int64")
print(df_clean["Age"])
```

Output:

```
0      22
1      38
2      26
3      35
4      35
...
886     27
887     19
888  <NA>
889     26
890     32
Name: Age, Length: 891, dtype: Int64
```

Step 3: Data Analysis

Display the first 5 rows of the dataset

```
data.head()
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Display dataset structure and data types

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch       891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object

dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Show the number of rows and columns in the dataset

```
data.shape
```

Output:

```
(891, 12)
```

Statistical Summary / Descriptors

```
display(HTML("<h3 style='text-align:center;'>Statistical Summary of Dataset</h3>"))
summary = df.describe()
display(summary)
```

Output:**Statistical Summary of Dataset**

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Step 4: Data Preprocessing

(HANDLING MISSING VALUES) Visualize and display missing values in each column

```
missing_count = data.isna().sum()
print("Missing values count per column:")
display(missing_count.to_frame(name="Missing Count"))
```

Output:

Missing values count per column:

Missing Count

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

Interactive widget to handle missing values in 'Age' using Mean/Median/Mode/Constant with preview and apply option

```
# INTERACTIVE NUMERIC MISSING VALUE IMPUTATION

feature = "Age"
display(HTML("<h3 style='text-align:center;'>INTERACTIVE NUMERIC MISSING VALUE IMPUTATION</h3>"))

method_dropdown = widgets.Dropdown(
    options=['Mean', 'Median', 'Mode', 'Constant'],
    description='Method'
)
constant_value = widgets.IntText(
    value=0,
    description='Constant'
)
apply_button = widgets.Button(
    description="Impute the Missing Values",
    button_style='success',
    layout=widgets.Layout(width='420px', height='40px')
)

output = widgets.Output()
imputation_done = False

# IMPORTANT: keep a copy of original data
df_preview = df_clean.copy()
nan_rows_fixed = df_preview[df_preview[feature].isna()].sample(5)
non_nan_rows_fixed = df_preview[df_preview[feature].notna()].sample(5)

sample_index = pd.concat([nan_rows_fixed, non_nan_rows_fixed]).index

def interactive_impute(change=None):

    global imputation_done

    with output:
        clear_output()
        temp_df = df_clean.copy()
        missing_before = temp_df[feature].isna().sum()
        print("The numeric feature 'Age' contains 177 missing values.")

        method = method_dropdown.value

        if method == "Mean":
            value = int(round(temp_df[feature].mean()))
            label = "Mean"

        elif method == "Median":
            value = int(round(temp_df[feature].median()))
            label = "Median"

        elif method == "Mode":
            value = int(temp_df[feature].mode()[0])
            label = "Mode"

        else:
            value = constant_value.value
            label = "Constant"

        after_series = temp_df[feature].fillna(value)

        result = pd.DataFrame({
            "Before_Imputation_Age": df_preview[feature],
            "After_Imputation_Age": after_series
        })

        # ALWAYS take NaN rows from original preview copy
        sample_table = result.loc[sample_index]
        sample_table.index = [''] * len(sample_table)

        # keep After column empty before clicking button
```

```

    if not imputation_done:
        sample_table["After_Imputation_Age"] = ""

    display(HTML(f"""
<div style="width:420px; text-align:center; margin-bottom:5px;"

<div style="color:#1d3557; font-size:15px; margin-bottom:3px;">
    {missing_before} NaN missing values will be replaced with {label} {value}
</div>

<div style="color:red; font-size:28px;">
    ↑
</div>

</div>
    """))

    display(apply_button)

    print("\nRandom 10 Sample Showing Imputation Effect")

    display(sample_table)

def apply_imputation(b):

    global imputation_done
    method = method_dropdown.value

    if method == "Mean":
        value = int(round(df_clean[feature].mean()))
        label = "Mean"

    elif method == "Median":
        value = int(round(df_clean[feature].median()))
        label = "Median"

    elif method == "Mode":
        value = int(df_clean[feature].mode()[0])
        label = "Mode"

    else:
        value = constant_value.value
        label = "Constant"

    df_clean[feature] = df_clean[feature].fillna(value)

    imputation_done = True

    with output:
        print(f"\nImputation finalized. Missing values in Age have been permanently filled using {label} method.")

    interactive_impute()

def reset_preview(change=None):
    global imputation_done
    imputation_done = False
    interactive_impute()

method_dropdown.observe(reset_preview, names='value')
constant_value.observe(reset_preview, names='value')

apply_button.on_click(apply_imputation)

display(method_dropdown, constant_value, output)

interactive_impute()

```

Output:

► INTERACTIVE OUTPUT (After Imputation)

INTERACTIVE NUMERIC MISSING VALUE IMPUTATION

Method: Mean

The numeric feature 'Age' contains 177 missing values.

177 NaN missing values will be replaced with Mean 30

Random 10 Sample Showing Imputation Effect

Before_Imputation_Age	After_Imputation_Age
<NA>	30
<NA>	30
<NA>	30
<NA>	30
<NA>	30
22	22
38	38
26	26
35	35
35	35

Imputation finalized. Missing values in Age have been permanently filled using Mean method.

4.2 Categorical Missing Value Imputation (Cabin & Embarked)

Interactive Output: Cabin (687 missing) filled with most frequent value "B96 B98". Embarked (2 missing) filled with most frequent value "S".

Interactive imputation of categorical missing values (Cabin & Embarked) using most frequent value

```
# FILLING CATEGORICAL MISSING VALUES WITH THE MOST FREQUENT VALUE

display(HTML("""
<h3 style="text-align:center;">
FILLING CATEGORICAL MISSING VALUES WITH THE MOST FREQUENT VALUE
</h3>
"""))

# CREATE CLEAN COPY
df_clean = df.copy()

# FIND MOST FREQUENT VALUES
most_frequent_embarked = df["Embarked"].mode()[0]
most_frequent_cabin = df["Cabin"].mode()[0]

# OUTPUT AREA
output = widgets.Output()

# BUTTONS
cabin_button = widgets.Button(
    description="Impute Missing Values for Cabin",
    button_style='success',
    layout=widgets.Layout(width='320px', height='40px')
)

embarked_button = widgets.Button(
    description="Impute Missing Values for Embarked",
    button_style='success',
    layout=widgets.Layout(width='320px', height='40px')
)

# FIXED SAMPLE TABLES (NO RESHUFFLE)
cabin_sample = df_clean[["Cabin"]].sample(6).copy()

embarked_nan = df_clean[df_clean["Embarked"].isna()][["Embarked"]]
embarked_non_nan = df_clean[df_clean["Embarked"].notna()][["Embarked"]].sample(4)
embarked_sample = pd.concat([embarked_nan, embarked_non_nan]).copy()

# STATE MEMORY
cabin_done = False
embarked_done = False

# DISPLAY FUNCTION
def display_tables():
    with output:
        clear_output()
        print("Missing values in Embarked:", df_clean["Embarked"].isna().sum())
        print("Missing values in Cabin:", df_clean["Cabin"].isna().sum())
        print("\nMost frequent value in Embarked:", most_frequent_embarked)
        print("Most frequent value in Cabin:", most_frequent_cabin)
        print("\nConcept Explanation:")
        print("""
Categorical missing values are commonly filled using the
```

most frequent category (also called the mode).

Reason:

Replacing missing values with the most frequent category preserves the distribution of the dataset and avoids introducing new artificial categories.

In this dataset:

Embarked missing values will be replaced with the most frequent port.

Cabin missing values will be replaced with the most frequent cabin value.

Once the button is pressed, the missing values will be filled and the dataset will be updated.

```
"""
```

```
    # CABIN
    display(cabin_button)
    cabin_table = cabin_sample.copy()
    if cabin_done:
        cabin_table["After_Cabin"] = cabin_table["Cabin"].fillna(most_frequent_cabin)
    else:
        cabin_table["After_Cabin"] = ""
        cabin_table.columns = ["Before_Cabin", "After_Cabin"]
        cabin_table.index = ["" ] * len(cabin_table)
        print("\n6 Random Samples Showing Cabin Imputation Effect")
        display(cabin_table)

    # EMBARKED
    display(embarked_button)
    embarked_table = embarked_sample.copy()
    if embarked_done:
        embarked_table["After_Embarked"] = embarked_table["Embarked"].fillna(most_frequent_embarked)
    else:
        embarked_table["After_Embarked"] = ""
        embarked_table.columns = ["Before_Embarked", "After_Embarked"]
        embarked_table.index = ["" ] * len(embarked_table)
        print("\n6 Random Samples Showing Embarked Imputation Effect")
        display(embarked_table)

# BUTTON FUNCTIONS
def fill_cabin(b):
    global cabin_done
    df_clean["Cabin"] = df_clean["Cabin"].fillna(most_frequent_cabin)
    cabin_done = True
    display_tables()

def fill_embarked(b):
    global embarked_done
    df_clean["Embarked"] = df_clean["Embarked"].fillna(most_frequent_embarked)
    embarked_done = True
    display_tables()

# BUTTON EVENTS
cabin_button.on_click(fill_cabin)
embarked_button.on_click(fill_embarked)

# DISPLAY
display(output)
display_tables()
```

Output:

► INTERACTIVE OUTPUT (After Imputation)

FILLING CATEGORICAL MISSING VALUES WITH THE MOST FREQUENT VALUE

Missing values in Embarked: 2
Missing values in Cabin: 687

Most frequent value in Embarked: S
Most frequent value in Cabin: B96

Concept Explanation:

Categorical missing values are commonly filled using the most frequent category (also called the mode).

Reason:

Replacing missing values with the most frequent category preserves the distribution of the dataset and avoids introducing new artificial categories.

6 Random Samples Showing Cabin Imputation Effect

Before_Cabin	After_Cabin
NaN	B96
C85	C85
NaN	B96
NaN	B96
E46	E46
NaN	B96

6 Random Samples Showing Embarked Imputation Effect

Before_Embarked	After_Embarked
NaN	S
NaN	S
S	S
C	C
Q	Q
S	S

Step 5: Encoding Categorical Variables

Interactive encoding of categorical features (Sex, Embarked, Cabin, Pclass) using Label, One-Hot, and Ordinal encoding

```
# ENCODING CATEGORICAL VARIABLES

display(HTML("<h3 style='text-align:center;'>Encoding Categorical Variables</h3>"))

# EXTRACT DECK LETTER FROM CABIN
if "Cabin" in df_clean.columns:
    df_clean["Cabin"] = df_clean["Cabin"].astype(str).str[0]

# TRACK ENCODED FEATURES
encoded_features = set()
before_snapshots = {}

# ENCODING METHOD DROPDOWN
encoding_method = widgets.Dropdown(
    options=["Label Encoding", "One-Hot Encoding", "Ordinal Encoding"],
    description="Encoding:"
)

# FEATURE MAP
feature_map = {
    "Label Encoding": ["Sex"],
    "One-Hot Encoding": ["Embarked", "Cabin"],
    "Ordinal Encoding": ["Pclass"]
}

# FEATURE DROPDOWN
encoding_feature = widgets.Dropdown(
    options=[],
    description="Feature:"
)

# OUTPUT
preview_output = widgets.Output()

# BUTTON
encode_button = widgets.Button(
    description="Encode",
```



```

        button_style='success',
        layout=widgets.Layout(width='220px', height='45px')
    )

# UPDATE FEATURE DROPDOWN
def update_feature_dropdown(change=None):
    method = encoding_method.value
    available = [f for f in feature_map[method] if f not in encoded_features]
    if len(available) == 0:
        encoding_feature.options = []
        encoding_feature.value = None
        encode_button.disabled = True
        encode_button.description = "No Feature Available"
        return
    encoding_feature.options = available
    encoding_feature.value = available[0]
    update_button()

encoding_method.observe(update_feature_dropdown, names="value")

# UPDATE BUTTON
def update_button():
    feature = encoding_feature.value
    if feature is None:
        encode_button.disabled = True
        encode_button.button_style = ''
        encode_button.description = "Encode"
        return
    encode_button.description = f"Encode {feature}"
    if feature in encoded_features:
        encode_button.disabled = True
        encode_button.button_style = ''
    else:
        encode_button.disabled = False
        encode_button.button_style = 'success'

encoding_feature.observe(lambda change: update_button(), names="value")
update_feature_dropdown()

# PREVIEW FUNCTION
def encode_toggle(method, feature):
    with preview_output:
        clear_output()
        if feature is None:
            return
        temp = df_clean.copy()
        if feature not in before_snapshots and feature in temp.columns:
            before_snapshots[feature] = temp[[feature]].head().copy()
        if feature in before_snapshots:
            before = before_snapshots[feature]
        else:
            return
        print("Selected Encoding Method:", method)
        print("Selected Feature:", feature)
        if feature in temp.columns:
            temp[feature] = temp[feature].astype(str)

    # LABEL ENCODING
    if method == "Label Encoding":
        print("\nLabel Encoding:")
        print("""Label Encoding converts categorical values into numbers.
For the feature 'Sex', the two categories are male and female.
Mapping used in this dataset:
female -> 0
male -> 1""")
        le = LabelEncoder()
        temp[feature] = le.fit_transform(temp[feature])
        after = temp[[feature]].head()

    # ONE HOT ENCODING
    elif method == "One-Hot Encoding":
        print("\nOne-Hot Encoding:")
        print("One-Hot Encoding creates separate binary columns for each category.")
        if feature == "Embarked":
            print("""
For 'Embarked', passengers boarded from three ports:
C (Cherbourg), Q (Queenstown), and S (Southampton).
Three columns are created: Embarked_C, Embarked_Q, Embarked_S.""")
            if feature == "Cabin":
                print("""
For 'Cabin', the deck letter is extracted first.
Possible decks include A, B, C, D, E, F, G and T.
One column is created for each deck such as Cabin_A, Cabin_B etc.""")
                if feature in temp.columns:
                    after = pd.get_dummies(temp[[feature]], prefix=feature).head()
                else:
                    after = pd.get_dummies(before, prefix=feature)
            if feature == "Embarked":

```

```

        expected_cols = ["Embarked_C", "Embarked_Q", "Embarked_S"]
        after = after.reindex(columns=expected_cols, fill_value='False')
    if feature == "Cabin":
        expected_cols = ["Cabin_A", "Cabin_B", "Cabin_C", "Cabin_D",
                        "Cabin_E", "Cabin_F", "Cabin_G", "Cabin_T"]
        after = after.reindex(columns=expected_cols, fill_value='False')

# ORDINAL ENCODING
else:
    print("\nOrdinal Encoding:")
    print("""Ordinal Encoding is used when categories have a natural ranking.
In the Titanic dataset, 'Pclass' represents passenger class:
1 = First Class, 2 = Second Class, 3 = Third Class.
The encoder assigns increasing integers:
1 → 0, 2 → 1, 3 → 2""")
    enc = OrdinalEncoder()
    temp[[feature]] = enc.fit_transform(temp[[feature]]).astype(int)
    after = temp[[feature]].head()

before = before.reset_index(drop=True)
after = after.reset_index(drop=True)
if feature in encoded_features:
    after_display = after
else:
    after_display = pd.DataFrame(
        ["" * after.shape[1]] * after.shape[0],
        columns=after.columns
    )
display(HTML(f"""
<div style="display:flex; gap:40px;">
    <div>
        <h4>Dataset Before Encoding</h4>
        {before.to_html(index=False)}
    </div>
    <div>
        <h4>Dataset After Encoding</h4>
        {after_display.to_html(index=False)}
    </div>
</div>
"""))

# FINALIZE ENCODING
def finalize_encoding(b):
    global df_clean
    feature = encoding_feature.value
    method = encoding_method.value
    if feature not in df_clean.columns:
        return
    if method == "Label Encoding":
        le = LabelEncoder()
        df_clean[feature] = le.fit_transform(df_clean[feature].astype(str))
    elif method == "One-Hot Encoding":
        encoded = pd.get_dummies(df_clean[feature], prefix=feature)
        df_clean = pd.concat([df_clean.drop(columns=[feature]), encoded], axis=1)
    else:
        enc = OrdinalEncoder()
        df_clean[[feature]] = enc.fit_transform(df_clean[[feature]]).astype(int)
    encoded_features.add(feature)
    encode_button.disabled = True
    encode_button.button_style = ''
    encode_toggle(method, feature)

# BUTTON CALLBACK
encode_button.on_click(finalize_encoding)

# DISPLAY
widgets.interact(
    encode_toggle,
    method=encoding_method,
    feature=encoding_feature
)

display(preview_output)
display(encode_button)

```

Output:

5.1 Label Encoding (Sex)

Interactive Output: Label Encoding applied to 'Sex' feature. female → 0, male → 1

► INTERACTIVE OUTPUT (After Encoding)

Label Encoding – Feature: Sex

Label Encoding converts categorical values into numbers.
For the feature 'Sex', the two categories are male and female.
Mapping used in this dataset:
female → 0
male → 1

Dataset Before Encoding

Sex
male
female
female
female
male

Dataset After Encoding

Sex
1
0
0
0
1

5.2 One-Hot Encoding (Embarked)

► INTERACTIVE OUTPUT (After Encoding)

One-Hot Encoding – Feature: Embarked

One-Hot Encoding creates separate binary columns for each category.
For 'Embarked', passengers boarded from three ports:
C (Cherbourg), Q (Queenstown), and S (Southampton).
Three columns are created: Embarked_C, Embarked_Q, Embarked_S.

Before Encoding

Embarked
S
C
S
S
S

After Encoding

Embarked_C	Embarked_Q	Embarked_S
False	False	True
True	False	False
False	False	True
False	False	True
False	False	True

5.3 One-Hot Encoding (Cabin)

► INTERACTIVE OUTPUT (After Encoding)

One-Hot Encoding – Feature: Cabin

For 'Cabin', the deck letter is extracted first.
Possible decks include A, B, C, D, E, F, G and T.
One column is created for each deck such as Cabin_A, Cabin_B etc.

Before Encoding

Cabin
B
C
B
C
B

After Encoding

Cabin_A	Cabin_B	Cabin_C	Cabin_D	Cabin_E	Cabin_F	Cabin_G	Cabin_T
False	True	False	False	False	False	False	False
False	False	True	False	False	False	False	False
False	True	False	False	False	False	False	False
False	False	True	False	False	False	False	False
False	True	False	False	False	False	False	False

5.4 Ordinal Encoding (Pclass)

► INTERACTIVE OUTPUT (After Encoding)

Ordinal Encoding – Feature: Pclass

Ordinal Encoding is used when categories have a natural ranking.
In the Titanic dataset, 'Pclass' represents passenger class:
1 = First Class, 2 = Second Class, 3 = Third Class.
The encoder assigns increasing integers:
1 → 0, 2 → 1, 3 → 2

Before Encoding

Pclass
3
1
3
1
3

After Encoding

Pclass
2
0
2
0
2

Step 6: Normalization / Feature Scaling

Interactive scaling of numerical features (Age, Fare) using Standard, MinMax, and Robust scalers

```
# INTERACTIVE FEATURE SCALING
display(HTML("<h3 style='text-align:center;'>INTERACTIVE FEATURE SCALING</h3>"))
scaled_features = set()
scale_feature = widgets.Dropdown(options=["Age", "Fare"], description="Feature:")
scale_method = widgets.Dropdown(options=["StandardScaler", "MinMaxScaler", "RobustScaler"], description="Scaler:")
scale_button = widgets.Button(description="Scale Age", button_style='success')
output = widgets.Output()

def update_button(change=None):
    feature = scale_feature.value
    if feature in scaled_features:
        scale_button.description = f"{feature} Already Scaled"
        scale_button.disabled = True
        scale_button.button_style = ''
        scale_button.style.button_color = "#BDBDBD"
        return
    scale_button.disabled = False
    scale_button.style.button_color = None
    if feature == "Age":
        scale_button.description = "Scale Age"
        scale_button.button_style = "success"
    else:
        scale_button.description = "Scale Fare"
        scale_button.button_style = "success"

scale_feature.observe(update_button, names="value")

def get_scaler():
    if scale_method.value == "StandardScaler":
        return StandardScaler()
    elif scale_method.value == "MinMaxScaler":
        return MinMaxScaler()
    else:
        return RobustScaler()

def preview(change=None):
    with output:
        clear_output()
        scaler_name = scale_method.value
        print("\nConcept Explanation:")
        print("Feature scaling transforms numerical values so that different features operate on comparable ranges.")
        if scaler_name == "StandardScaler":
            print("""
StandardScaler (Standardization)
Formula:  $z = (x - \mu) / \sigma$ 
Where:  $x$  = original value,  $\mu$  = mean,  $\sigma$  = standard deviation""")
            elif scaler_name == "MinMaxScaler":
                print("""
MinMaxScaler (Normalization)
Formula:  $x' = (x - x_{min}) / (x_{max} - x_{min})$ """)
            else:
                print("""
RobustScaler
Formula:  $x' = (x - \text{median}) / \text{IQR}$ """)

def run_scaling(b):
    with output:
        clear_output()
        feature = scale_feature.value
        scaler_name = scale_method.value
```

```

data = df_clean[feature].dropna()
reshaped = data.values.reshape(-1,1)
sc = get_scaler()
scaled = sc.fit_transform(reshaped).flatten()
df_clean[f"Scaled_{feature}"] = np.nan
df_clean.loc[data.index, f"Scaled_{feature}"] = scaled
scaled_features.add(feature)
fig, ax = plt.subplots(1,2,figsize=(10,4))
sns.histplot(data, bins=20, ax=ax[0], kde=True)
ax[0].set_title("Before Scaling")
sns.histplot(scaled, bins=20, ax=ax[1], kde=True)
ax[1].set_title(f"After {scaler_name}")
plt.show()
table = pd.DataFrame({f"Actual_{feature}": data.values.astype(int), f"Scaled_{feature}": scaled.round(3)})
sample_table = table.sample(30)
sample_table.index = [""] * len(sample_table)
display(HTML("<b>Sample Values Showing Scaling Effect</b>"))
display(HTML(f"""
<div style="height:220px; overflow-y:auto; border:1px solid #ccc;">
{sample_table.to_html(index=False)}
</div>"""))
print(f"\nScaled {feature} values saved in df_clean as column: Scaled_{feature}")

scale_method.observe(preview, names="value")
scale_feature.observe(preview, names="value")
scale_button.on_click(run_scaling)
display(scale_feature)
display(scale_method)
display(scale_button)
display(output)
preview()

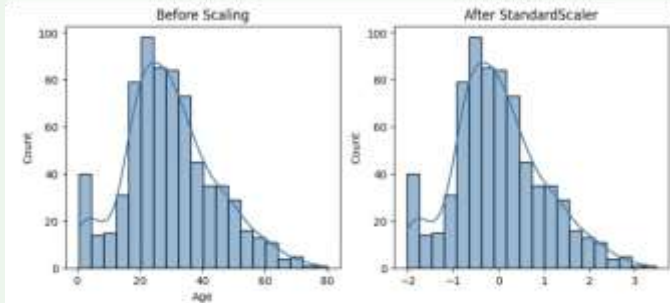
```

Output:

► INTERACTIVE OUTPUT (After Scaling — Age with StandardScaler)

Concept Explanation:
Feature scaling transforms numerical values so that different features operate on comparable ranges. This prevents machine learning algorithms from being biased toward features with larger magnitudes.
StandardScaler (Standardization)
This method transforms the feature so that the resulting values have a mean of 0 and a standard deviation of 1.
Formula:

$$z = (x - \mu) / \sigma$$
Where:
x = original value
μ = mean
σ = standard deviation



Sample Values Showing Scaling Effect

Actual_Age	Scaled_Age
22	-0.530
38	0.572
26	-0.254
35	0.365
54	1.674
2	-1.908
27	-0.185
14	-1.081

Scaled Age values saved in df_clean as column: Scaled_Age

Step 7: Outlier Detection

Interactive outlier detection for numerical features (Age, Fare) using IQR and Z-Score methods

```

# INTERACTIVE OUTLIER DETECTION
display(HTML("<h3 style='text-align:center;'>INTERACTIVE OUTLIER DETECTION</h3>"))
outlier_feature = widgets.Dropdown(options=["Age","Fare"], description="Feature:")
outlier_method = widgets.Dropdown(options=["IQR","Z-Score"], description="Method:")
output = widgets.Output()

def detect_outlier(feature, method):
    with output:
        clear_output(wait=True)
        data = df_clean[feature].dropna()

```

```

if method == "IQR":
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    if feature == "Fare": multiplier = 4.5
    else: multiplier = 2.3
    lower = Q1 - multiplier * IQR
    upper = Q3 + multiplier * IQR
    outliers = data[(data < lower) | (data > upper)]
    explanation = f"IQR (Interquartile Range) Method"
Formula: IQR = Q3 - Q1
Lower Bound = Q1 - {multiplier} * IQR
Upper Bound = Q3 + {multiplier} * IQR"""
else:
    z_scores = np.abs(zscore(data))
    outliers = data[z_scores > 3]
    explanation = """Z-Score Method
Formula: z = (x - μ) / σ
Rule used: |z| > 3 → Outlier"""
    print("Selected Feature:", feature)
    print("Detection Method:", method)
    print("Total Data Points:", len(data))
    print("Number of Outliers Detected:", len(outliers))
    print("\nConcept Explanation:")
    print(explanation)
    plt.figure(figsize=(8,4))
    sns.boxplot(x=data)
    if len(outliers) > 0:
        plt.scatter(outliers, [0]*len(outliers), color="red", label="Detected Outliers")
        plt.legend()
    plt.title(f"Outlier Detection for {feature} using {method}")
    plt.show()

widgets.interact(detect_outlier, feature=outlier_feature, method=outlier_method)
display(output)

```

Output:

► INTERACTIVE OUTPUT (Age — IQR Method)

Selected Feature: Age
 Detection Method: IQR
 Total Data Points: 891
 Number of Outliers Detected: 1

Concept Explanation:

IQR (Interquartile Range) Method

The IQR method detects outliers by examining the spread of the middle 50% of the dataset.

Formula:
 $IQR = Q3 - Q1$

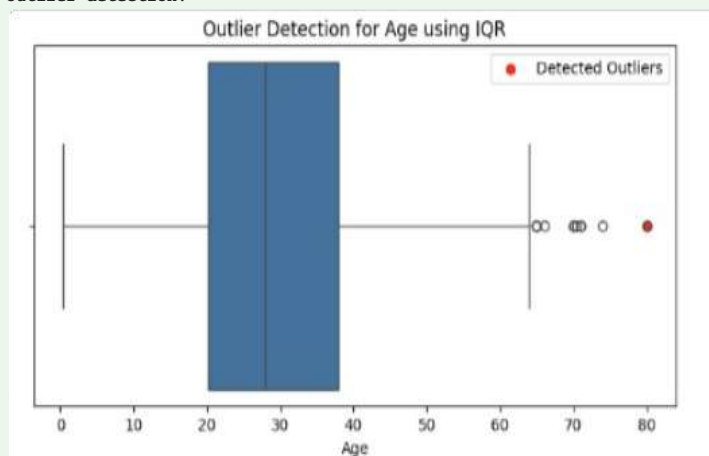
Where:
 $Q1 = 25\text{th percentile}$
 $Q3 = 75\text{th percentile}$

Outlier limits:

$\text{Lower Bound} = Q1 - 2.3 \times IQR$
 $\text{Upper Bound} = Q3 + 2.3 \times IQR$

Traditionally the multiplier used is $1.5 \times IQR$. However, datasets such as the Titanic Fare distribution are highly skewed. Using 1.5 may label many moderate values as outliers.

Therefore a relaxed multiplier ($2.3 \times IQR$) is used to focus on more extreme deviations and reduce excessive outlier detection.



Output:

► INTERACTIVE OUTPUT (Fare — Z-Score Method)

Selected Feature: Fare
Detection Method: Z-Score
Total Data Points: 891
Number of Outliers Detected: 20

Concept Explanation:
Z-Score Method

The Z-Score method determines how far a data point lies from the mean of the dataset in terms of standard deviations.

Formula:

$$z = (x - \mu) / \sigma$$

Where:

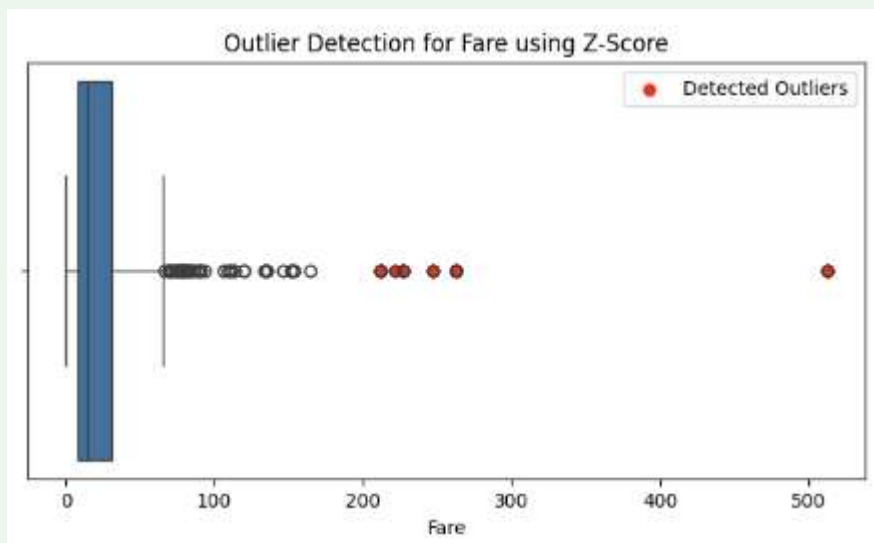
x = data value
 μ = mean of dataset
 σ = standard deviation

Rule used:

$|z| > 3 \rightarrow \text{Outlier}$

This means the value is more than three standard deviations away from the mean.

This approach works best when the data distribution is approximately normal.



Step 8: Feature Engineering

```
# Create a new feature 'FamilySize' as SibSp + Parch + 1
```

```
df_family = df_clean.copy()
df_family["FamilySize"] = df_family["SibSp"] + df_family["Parch"] + 1
print("FamilySize feature created:")
display(df_family[["SibSp", "Parch", "FamilySize"]].head())
```

Output:

FamilySize feature created:

	SibSp	Parch	FamilySize
0	1	0	2
1	1	0	2
2	0	0	1
3	1	0	2
4	0	0	1

```
# Create a new feature 'IsAlone' where 1 indicates traveling alone
```

```
df_isalone = df_family.copy()
df_isalone["IsAlone"] = (df_isalone["FamilySize"] == 1).astype(int)
print("IsAlone feature created:")
display(df_isalone[["IsAlone"]].head())
```

Output:

IsAlone feature created:

	IsAlone
0	0
1	0
2	1
3	0
4	1

Add 'FamilySize' and 'IsAlone' features to the dataset

```
df_clean["FamilySize"] = df_clean["SibSp"] + df_clean["Parch"] + 1
df_clean["IsAlone"] = (df_clean["FamilySize"] == 1).astype(int)
display(df_clean[["FamilySize", "IsAlone"]].head())
```

Output:

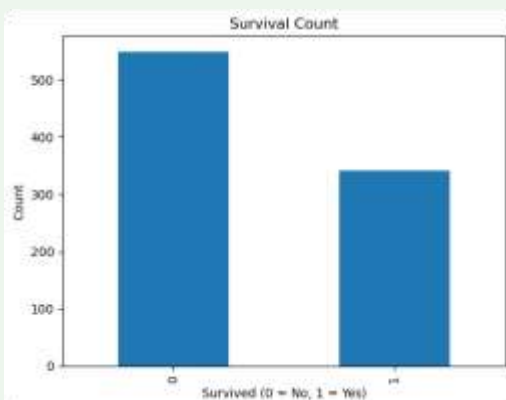
	FamilySize	IsAlone
0	2	0
1	2	0
2	1	1
3	2	0
4	1	1

Step 9: Visualization

(UNIVARIATE VISUALIZATION) Plot the count of passengers who survived vs. did not survive

```
plt.figure()
df_clean["Survived"].value_counts().sort_index().plot(kind="bar")
plt.title("Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```

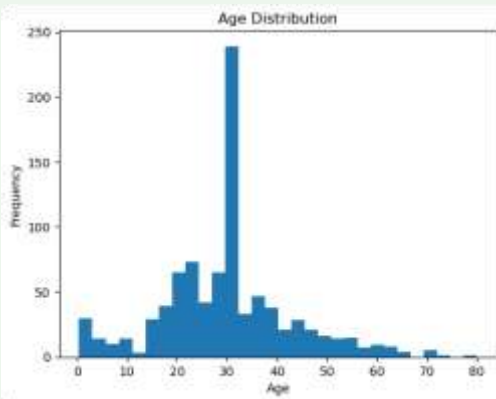
Output:



Plot the distribution of passenger ages

```
plt.figure()
plt.hist(df_clean["Age"], bins=30)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

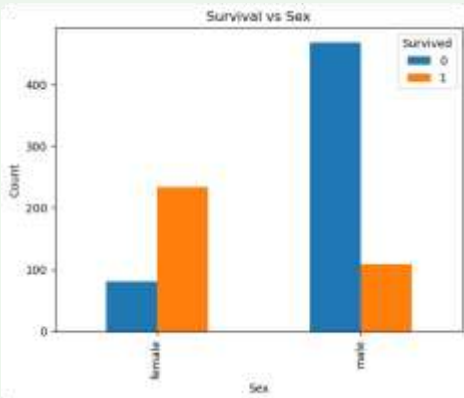

Output:



```
# (BIVARIATE VISUALIZATION) Plot a grouped bar chart of survival counts by sex
```

```
plt.figure()
df_clean.groupby(["Sex", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Sex")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()
```

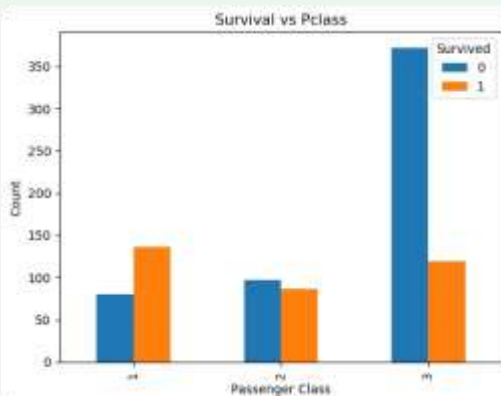
Output:



```
# Plot a grouped bar chart of survival counts by passenger class
```

```
plt.figure()
df_clean.groupby(["Pclass", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Pclass")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```

Output:

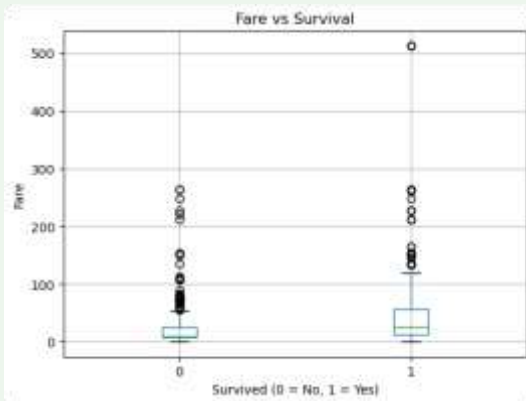


```
# Compare Fare distribution by survival status using a box plot
```

```
plt.figure()
df_clean.boxplot(column="Fare", by="Survived")
plt.title("Fare vs Survival")
plt.suptitle("")
```

```
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Fare")
plt.show()
```

Output:



Step 10: Data Comparison — Raw vs Processed

Displays side-by-side comparison of raw vs processed dataset (first 10 rows)

```
# BEFORE VS AFTER DATA COMPARISON
def compare_datasets():
    print("Raw Dataset vs Processed Dataset (first 10 samples)")
    raw = df_raw.head(10).copy()
    processed = df_clean.head(10).copy()
    if "Age" in raw.columns:
        raw["Age"] = raw["Age"].round().astype("Int64")
    if "Age" in processed.columns:
        processed["Age"] = processed["Age"].round().astype("Int64")
    display_html(f"""
<div style="display: flex; gap: 40px; align-items: flex-start;">
    <div style="background: #ffe5b4; padding: 15px; border-radius: 10px; width: 50%;">
        <h3 style="text-align: center;">Raw Dataset</h3>
        <div style="overflow-x: auto;">{raw.to_html(index=False)}</div>
    </div>
    <div style="background: #d8f3dc; padding: 15px; border-radius: 10px; width: 50%;">
        <h3 style="text-align: center;">Processed Dataset</h3>
        <div style="overflow-x: auto;">{processed.to_html(index=False)}</div>
    </div>
</div>""", raw=True)
compare_datasets()
```

